Kafka Stream: SerDes (Serializer Deserializer)

Monday, 29 June 2026 3:23 PM

Recap: Source Syntax

```
//KStream
 KStream<String, String> orders = builder.stream("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String())
                                    );

//KTable
  KTable<String, String> orders = builder.table("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String()),
                                    Materialized
                                        .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                        .withKeySerde(Serdes.String())
                                        .withValueSerde(Serdes.String())
                                    );
//GlobalKTable
GlobalKTable<String, String> countries = builder.globalTable("source-topic-name",
                                            Consumed.with(Serdes.String(), Serdes.String()),
                                            Materialized
                                                .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                                .withKeySerde(Serdes.String())
                                                .withValueSerde(Serdes.String())
                                            );
```

SerDes:

SerDes = Serializer + Deserializer combined into 1 object

SERIALIZER: Java Object ----> bytes

DESERIALIZER: bytes ----> Java Object

SERDE: Both in one package

What's the need?

Kafka stores data in bytes
RocksDB stores data in bytes

So whenever we need to:
- Read data from Kafka topic or RocksDB, we need Deserialization (bytes to Java Object) and
- Write data to Kafka topic or RocksDB, we need Serialization (Java Object to bytes)

So 5 places within topology where we need SerDes:

- Source: we need to read from Kafka topic, so we need Deserialization.
- Sink: we need to write to Kafka topic, so we need Serialization.
- Repartition (Sub-topology): we need to write to Kafka topic, so we need Serialization.
- RocksDB Write:
  - Like when we use KTable, GlobalKTable we need Serialization.
  - Or processor nodes like count etc, which need to store data in RocksDB.
- RocksDB Read:
  - Like when we do JOIN, we read from RocksDB, so we need Deserialization.

Built-in Serdes:

Kafka Streams provides these out of the box:

Serdes.String()	String <---> bytes
Serdes.Long()	Long <---> bytes
Serdes.Integer()	Integer <---> bytes

Serdes.Double()	Double <---> bytes
Serdes.ByteArray()	byte[] <---> bytes
Serdes.Bytes()	Bytes object <---> bytes
Serdes.UUID()	UUID <---> bytes
Serdes.Float()	Float <---> bytes
Serdes.Short()	Short <---> bytes
Serdes.Boolean()	Boolean <---> bytes

SerDes Priority:

Lowest Priority:

Default SerDes, set in application.properties once. It will be applied everywhere.

application.properties

```
spring.kafka.streams.properties.default.key.serde=org.apache.kafka.common.serialization.Serdes$StringSerde
spring.kafka.streams.properties.default.value.serde=org.apache.kafka.common.serialization.Serdes$StringSerde
```

Without default SerDes, we must specify key and value SerDes everywhere.

```
//KStream
 KStream<String, String> orders = builder.stream("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String())
                                    );

//KTable
  KTable<String, String> orders = builder.table("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String()),
                                    Materialized
                                          .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                          .withKeySerde(Serdes.String())
                                          .withValueSerde(Serdes.String())
                                    );
//GlobalKTable
GlobalKTable<String, String> countries = builder.globalTable("source-topic-name",
                                              Consumed.with(Serdes.String(), Serdes.String()),
                                              Materialized
                                                  .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                                  .withKeySerde(Serdes.String())
                                                  .withValueSerde(Serdes.String())
                                              );
```

Now, let's look at the same source types without explicitly specifying the SerDes.

```
//KStream
 KStream<String, String> orders = builder.stream("source-topic-name");

//KTable
  KTable<String, String> orders = builder.table("source-topic-name",
                                            Materialized.<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-
                                            store-name")
                                          );
//GlobalKTable
GlobalKTable<String, String> countries = builder.globalTable("source-topic-name",
```

```
                                        Materialized.<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-
                                        store-name")
                                            );
```

Highest Priority:

Explicitly providing the SerDes.

1. Source:

```
KStream<String, String> orders = builder.stream("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String())
                                    );

//KTable
 KTable<String, String> orders = builder.table("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String()),
                                    Materialized
                                            .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                            .withKeySerde(Serdes.String())
                                            .withValueSerde(Serdes.String())
                                    );
//GlobalKTable
GlobalKTable<String, String> countries = builder.globalTable("source-topic-name",
                                            Consumed.with(Serdes.String(), Serdes.String()),
                                            Materialized
                                                .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                                .withKeySerde(Serdes.String())
                                                .withValueSerde(Serdes.String())
                                            );
```

2. Sink:

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kstreamDemo(StreamsBuilder builder) {

            KStream<String, String> orders = builder.stream("order-events",
                                                Consumed.with(Serdes.String(), Serdes.String())
                                                );

             orders.peek((key, value) ->
                            System.out.println("key=" + key + " value=" + value));

             orders.to("order-events-output",
                                            Produced.with(Serdes.String(), Serdes.String()));
    }
}
```

3. RocksDB Write:

```
//KTable
 KTable<String, String> orders = builder.table("source-topic-name",
                                    Consumed.with(Serdes.String(), Serdes.String()),
                                    Materialized
                                            .<String, String, KeyValueStore<Bytes, byte[]>>as("RocksDB-store-name")
                                            .withKeySerde(Serdes.String())
                                            .withValueSerde(Serdes.String())
                                    );
```

likewise.

Now, doubt might have come that: How to handle the custom Java Objects?

Use: JsonSerde

This is already built-in framework. But lets see, if we have to write same JsonSerde, how to write it:

1. Serializer

```
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Serializer;

public class JsonSerializer<T> implements Serializer<T> {

        private final ObjectMapper objectMapper = new ObjectMapper();

        @Override
        public byte[] serialize(String topic, T data) {
                if (data == null) {
                    return null;
                }
                try {
                        return objectMapper.writeValueAsBytes(data);
                }
                catch (Exception e) {
                        throw new RuntimeException("Error serializing", e);
                    }
        }
}
```

Generic type here, because it can handle any object like:
Order
Payment
Etc.

2. Deserializer

```
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Deserializer;

public class JsonDeserializer<T> implements Deserializer<T> {

        private final ObjectMapper objectMapper = new ObjectMapper();

        private final Class<T> targetType;

            public JsonDeserializer(Class<T> targetType) {
                    this.targetType = targetType;
            }

            @Override
            public T deserialize(String topic, byte[] data) {
                    if (data == null) {
                        return null;
                    }
                    try {
                            return objectMapper.readValue(data, targetType);
                    } catch (Exception e) {
                            throw new RuntimeException("Error deserializing", e);
                    }
            }
}
```

Our Serializer is generic means it can accept any Object.
So during Deserialization, we need to know the object type in which we need to convert.

3. SerDes (Object which has both Serializer and Deserializer)

```
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;

public class JsonSerde<T> implements Serde<T> {

    private final JsonSerializer<T> serializer;
    private final JsonDeserializer<T> deserializer;

    public JsonSerde(Class<T> targetType) {
        this.serializer = new JsonSerializer<>();
        this.deserializer = new JsonDeserializer<>(targetType);
    }

    @Override
    public JsonSerializer<T> serializer() {
            return serializer;
    }

    @Override
```

Like this we can create our own Custom Serde.

Note:
JsonSerde class already present in framework, we don't need to create this, its just for understanding.

```
    public JsonDeserializer<T> deserializer() {
        return deserializer;
    }
}
```

Now lets use this:

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kstreamDemo(StreamsBuilder builder) {

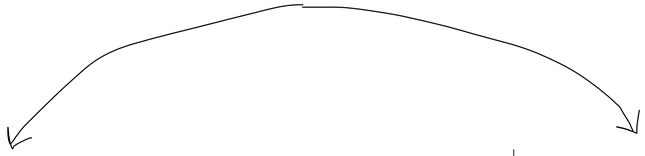

                Serde<Order> orderSerde = new JsonSerde<>(Order.class);

               KStream<String, Order> orders = builder.stream("order-events",
                                                Consumed.with(Serdes.String(), orderSerde)
                                                );


                orders.filter((key, order) -> order.getTotalAmount() > 100)
                        .to("order-events-output",
                                                Produced.with(Serdes.String(), orderSerde));
    }
}
```

Now 1 more doubt might have come:

What will happen if Serialization, Deserialization fails in Kafka Stream?



Deserialization fails

UseCase1: Wrong serde type

Topic has: {"orderId": "001", "amount": 150 } (JSON string)
We read: Consumed.with(Serdes.String(), Serdes.Long())

Result: Tries to read JSON bytes as 8-byte Long --> EXCEPTION

UseCase2: Corrupted data

Topic has: some garbage bytes that aren't valid JSON
We read: Consumed.with(Serdes.String(), new JsonSerde<>(Order.class))

Result: Jackson can't parse --> JsonParseException --> EXCEPTION

3 ways to handle it:

- LogAndFailExceptionHandler (Default)
  - It will log the error
  - Throws the exception
  - Stream thread dies
  - If all thread dies, application dies

application.properties

```
spring.kafka.streams.properties[default.deserialization.exception.handler]=
                org.apache.kafka.streams.errors.LogAndFailExceptionHandler
```

- LogAndContinueExceptionHandler
  - It will log the error
  - Skip the bad record

Serialization fails

UseCase 1: Wrong output serde
We have: Long value
We write: Produced.with(Serdes.String(), Serdes.String())

Result: Tries to serialize Long as String --> ClassCastException

UseCase 2: Object can't be serialized
We have: Order object with a circular reference
We write: JsonSerde<>(Order.class)

Result: Jackson can't serialize → JsonMappingException

2 ways to handle it:

- DefaultProductionExceptionHandler **(Default)**
  - Always Fail
  - Stream thread dies
  - If all thread dies, application dies

application.properties

```
spring.kafka.streams.properties[default.production.exception.handler]=
                org.apache.kafka.streams.errors.DefaultProductionExceptionHandler
```

We don't need to write below code, its just for understanding, what internally it might be doing.

- Continue processing next record
- Bad record is lost forever

application.properties

```
spring.kafka.streams.properties[default.deserialization.exception.handler]=
                    org.apache.kafka.streams.errors.LogAndContinueExceptionHandler
```

- Custom handler

  Our business logic, how we want to handle the bad request:

  - Either continue

  or

  - Fast fail

  or

  - Insert in Dead Letter Queue topic

```
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.streams.errors.DeserializationExceptionHandler;
import org.apache.kafka.streams.processor.ProcessorContext;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

public class CustomDeserializationHandler implements DeserializationExceptionHandler {

    private long maxErrors;
    private static final AtomicLong errorCount = new AtomicLong(0);

    @Override
    public DeserializationHandlerResponse handle(ProcessorContext context,
                                                 ConsumerRecord<byte[], byte[]> record,
                                                 Exception exception) {

            long errors = errorCount.incrementAndGet();

            // STRATEGY: skip bad records, if limit not crossed
            If(errors > maxErrors) {
                return DeserializationHandlerResponse.FAIL;
            }

            return DeserializationHandlerResponse.CONTINUE;
    }

    @Override
    public void configure(Map<String, ?> configs) {
            // we can read or update stream config properties here if needed
            Object value = configs.get("stream.max.deserialization.errors");
            maxErrors = Long.parseLong(value.toString());
            if (value != null) {
                    maxErrors = Long.parseLong(value.toString());
            } else {
                    maxErrors = 100;
            }
    }
}
```

application.properties

```
spring.kafka.streams.properties[default.deserialization.exception.handler]=
                    com.eda.streams.errorhandling.CustomDeserializationHandler
```

```
public class DefaultProductionExceptionHandler implements ProductionExceptionHandler {

    @Override
    public ProductionExceptionHandlerResponse handle(ProducerRecord<byte[], byte[]> record,
                                                     Exception exception) {

            return ProductionExceptionHandlerResponse.FAIL;
    }
}
```

- Custom Handler

```
package com.eda.streams.errorhandling;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.streams.errors.ProductionExceptionHandler;
import java.util.Map;

public class CustomProductionHandler implements ProductionExceptionHandler {

    @Override
    public ProductionExceptionHandlerResponse handle(ProducerRecord<byte[], byte[]> record,
                                                     Exception exception) {

        if (exception.getClass().getSimpleName().contains("RecordTooLarge")) {
                return ProductionExceptionHandlerResponse.CONTINUE;
        }
        return ProductionExceptionHandlerResponse.FAIL;
    }

    @Override
    public void configure(Map<String, ?> configs) {

    }
}
```

application.properties

```
spring.kafka.streams.properties[default.production.exception.handler]=
                    com.eda.streams.errorhandling.CustomProductionHandler
```